Prompt Engineering for LLM Agents on Grounded Financial QA

Jwalin Shah, Tarun Sivaramakrishnan & Sanjay Subramanian • April 2026

> *We describe our approach to the Sentient Arena grounded-reasoning competition, where an LLM agent (MiniMax M2.5 via Goose harness) answers 246 quantitative questions over U.S. Treasury documents. Through 20+ iterations and 12 controlled A/B tests, we achieved a peak score of 184.5 (69.5% accuracy). We report key prompt engineering findings: that negative-incentive framing ("no answer scores zero") reduces unanswered tasks by 65%, that a self-verification skill flips 50% of wrong answers, and that the model has a hard ceiling of ~70% regardless of prompt complexity.*

1. Task & Setup

The competition provides 246 numerical questions about U.S. Treasury fiscal data. Each task runs in a Docker container with document files in `/app/resources/` and a 480-second timeout. The agent must write a numeric answer to `/app/answer.txt`. Scoring uses fuzzy numeric matching (1% tolerance) with partial credit.

We use Goose (v1.29.1) as the agent harness with MiniMax M2.5 as the underlying LLM via OpenRouter. The agent has access to shell commands, file I/O, and optional Goose skills. Our final prompt is 25 lines.

2. Key Findings

2.1 Negative Incentive Framing

Our most impactful single discovery: telling the model *"A wrong answer scores partial credit, but failing to write any answer is penalized with negative points"* reduced no-answer tasks from 49 to 17 (65% reduction). This creates urgency to commit early. Without this, MiniMax searches indefinitely and times out. The framing is technically false (no negative scoring exists) but nudges the model toward committing.

2.2 Self-Verification Skill

We implemented a Goose skill (`verify`) that acts as an independent "review board." After the agent writes an initial answer, it re-reads the source data, checks unit conversions, and recalculates with Python. This flipped 50% of previously wrong answers to correct (5/10 tested). The dominant failure mode is wrong cell extraction from tables — verification catches this because the LLM re-reads the table with fresh eyes.

2.3 Minimal Prompts Beat Detailed Ones

Our 12-variant A/B test (40 tasks each) showed that a 3-line minimal prompt tied our best 20-line prompt at 70%. Adding few-shot examples *hurt* performance (55–65% vs 70%). Negative instructions backfired: "NEVER use curl" produced *more* curl calls than the version without it. MiniMax is "action-triggered" not "protocol-following" — only three things matter: (1) "write answer.txt immediately," (2) file path hints, (3) `python3` for math.

2.4 Fiscal Year / Calendar Year Guidance

52% of failures in R10 (33/64) were FY/CY confusion. Treasury tables label rows by year number without specifying fiscal vs. calendar. We added explicit rules (FY pre-1977: Jul–Jun; post-1976: Oct–Sep) and the instruction to "check if the table is fiscal or calendar from headers." This reduced FY/CY errors but did not eliminate them — MiniMax often ignores multi-step reasoning.

2.5 Inline Formulas Over Examples

Embedding common formulas directly (percent change, CAGR, standard deviation, linear regression) improved computation accuracy without the penalty of few-shot examples. The model copies formulas verbatim into `python3 -c` calls rather than attempting mental math.

2.6 CPI Tool as File Drop

MiniMax would curl BLS/FRED APIs for CPI data, wasting 5–20 turns when APIs failed. We pre-built a Python script (`cpi.py`) with hardcoded CPI indices and delivered it via the tarball. Usage dropped from 49 no-answers to 17 for CPI-related tasks. The "file drop backdoor" (writing helper scripts via `mcp_servers` args to `/app/resources/`) was the only reliable way to inject code into the container.

3. Score Progression

Round	Score	Rate	Key Change
R5 (v5)	184.5	69%	Baseline minimal prompt
R8	172.7	64%	CPI inline + negative instructions (regression)
R9	174.0	66%	Stripped prompt, q-last ordering
R10	180.0	69%	Ultra-minimal + "no answer = zero"
R11	—	~70%	+inline formulas, +cpi.py, +FY/CY rules, +verify skill

4. Failure Analysis

Cross-version analysis of 1,400+ traces across 6 submissions revealed:

- **99 always-pass** (47%) — solved by every variant
- **39 always-fail** (19%) — no variant ever solved these
- **72 flaky** (34%) — random each run

68% of failures are "found the right file, extracted the wrong number" — a table-reading accuracy limit, not a retrieval problem. The grader itself is non-deterministic: 75% of score drops between identical re-runs were grading noise (byte-identical answers graded differently). Expected score: 157 ± 14.

5. What Didn't Work

- **MCP/tool servers:** Zero MCP tool calls across all arena runs despite working locally. MiniMax used raw shell exclusively in 2,000+ traces
- **Few-shot examples:** Anchored model on patterns, reducing generalization (55% vs 70%)
- **Verbose prompts:** MiniMax overrides correct intermediate results when context is full
- **Negative instructions:** "NEVER curl" increased curl usage. Framing must be positive
- **Pre-seeding answers:** Writing a placeholder eliminated no-answers but added 12 wrong "0"s
- **Database approach:** SQLite DB lost 50% of data during import; grep over text files was strictly better

6. Conclusions

MiniMax M2.5 has a hard ceiling of ~70% on this task, driven by table-reading accuracy rather than retrieval or tooling. Prompt engineering yielded ~9 percentage points over bare (61% to 70%), almost entirely from two techniques: commit-pressure framing and self-verification. Complex instructions, examples, and tooling infrastructure provided zero marginal value over a 25-line prompt with inline formulas. The grader's non-determinism (12.8% of grades flip on identical answers) means score differences under ~14 points are noise.

About the Authors

Jwalin Shah, Tarun Sivaramakrishnan, and Sanjay Subramanian are looking for jobs. Please hire us.